STUDENT INFORMATION MANAGEMENT SYSTEM WITH MOODLE INTEGRATION

BY

CHARLES HANGOMA (2021414469) & CHITUNDU MILIMBO (2020992574)

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT OF A BACHELOR OF COMPUTER SCIENCE DEGREE

THE UNIVERSITY OF ZAMBIA

October 2026

COPYRIGHT

All rights reserved. No part of this material may be reproduced, stored in any retrieval system, or transmitted in any form by any means. Except in the case of brief quotations embodied in critical reviews and other non-commercial uses permitted by the copyright law of the author, Charles Hangoma & Chitundu Milimbo or the University of Zambia in that regard.

DECLARATION

I, Charles Hangoma & Chitundu Milimbo do hereby declare that this report is my own original work and has not been submitted to any other college, institution or university other than the University of Zambia.

Name: …….

Sign: ……

Date: ……

APPROVAL

This report, by Charles Hangoma & Chitundu Milimbo has been approved as partial fulfilment of the requirements for the award of a Bachelor of Science in Computer Science Degree by the University of Zambia.

Student

Name: …….

Signature: ……..

Date: ……..

Supervisor

Name: …….

Signature: ……..

Date: ……..

ACKNOWLEDGEMENTS

This research would not have been possible without considerable information and facts gotten from other researchers and sources that have researched extensively on communication and to this the researcher duly acknowledges their input to the study.

My deepest appreciation goes to my supervisor, Prof. Jackson Phiri for taking the patience of reading through my work and making constructive criticisms and useful suggestions that helped me to stay focused from the beginning of this work till the end. I am highly indebted to my parents, for their constant financial, material and moral support without which this research would not have been made possible; and to all my brothers, sisters and family for their love, encouragement and prayers which gave me the strength to go on at every stage.

Secondly, I wish to extend a heartfelt thank you to all my incredibly special friends whose discussions and ideas in one way or another enlightened me while carrying out this research. Finally, I would like to say special thanks to God Almighty for giving me his grace throughout this period and providing me with both the intellectual and physical capacities and the opportunity to live up to the completion of this research. To all those whose names have not been mentioned here, know that am grateful to you.

DEDICATION

To my parents, Mr xxxxxxxxxx and Mrs xxxxxxxxxx, for their financial support throughout this sxxxxxxxxxxxxxxxxxxxxxxxxxxxxx ddddddddddddddddddddddddddddd fffffff fffffffffffffffffff ggggggggggggggggggggggg hhhhhh

ABSTRACT

Communication is a very crucial and significant element in an organization, and it is necessary for creating collaboration within the work environment that have effects on organizational performance and decision making. This study on effects of effective communications on organizational performance using the systems theory is thus essential. The objectives of this study were to firstly identify the factors affecting effective communication based on the systems theory and secondly to devise a communication model that addresses these factors to improve organisational performance. A quantitative approach was used to describe and analyse data collected for the study from 88 respondents. A questionnaire was used to identify factors affecting effective communication based on systems theory. Descriptive analysis and Pearson's correlation were used to analyse the results. The results for H_1 showed a Pearson correlation coefficient of 0.642 which is statistically insignificant if the (sig. 2-tailed) < 0.05 indicating that there was no statistical significant relationship between management and channel of communication used. However, the results from H_2 indicated that there was a relationship between Channel of communication used and effective communication with a Pearson correlation coefficient of 0.041 at (sig. 2-tailed) < 0.05 between communication channel variable; channel normally used and effective communication variable; receives information from preferred sources. Lastly the correlation results from H_3 showed that effective communication has a positive effect on organizational performance at (sig. 2-tailed) < 0.05 indicating a Pearson's correlation coefficient of 0.012.

Keywords: Effective, Communication, Performance, Systems theory, Organisation

TABLE OF CONTENTS

COPYRIGHT....i
DECLARATION....ii
APPROVAL....iii
ACKNOWLEDGEMENTS....iv
DEDICATION....v
ABSTRACT....vi
TABLE OF CONTENTS....vii
LIST OF TABLES....x
LIST OF FIGURES....xi
LIST OF ABBREVIATIONS....xii
1 INTRODUCTION AND BACKGROUND....1
1.1 Introduction....1
1.2 Background....1
1.3 Statement of the Problem....1
1.4 Aim of the Study....1
1.5 Research Objectives....1
1.6 Research Questions....1
1.7 Significance of the Study....2
1.8 Scope of the study....2
1.9 Organization of the Report....2
1.10 Chapter Summary....2
2 LITERATURE REVIEW....3
2.1 Introduction....3
2.2 Background to the Study....3
2.2.1 Sub Thematic Area 1....3
2.2.2 Sub Thematic Area 2....3
2.2.3 Sub Thematic Area 3....3
2.3 Thematic Area 1 related to objective 1....3
2.3.1 Sub Thematic area 1....3
2.3.2 Sub Thematic area 2....3
2.3.3 Sub Thematic area n....3
2.4 Thematic Area 2 related to objective 2....4
2.4.1 Sub Thematic area 1....4
2.4.2 Sub Thematic area 2....4
2.4.3 Sub Thematic area n....4
2.5 Thematic Area n related to objective n....4

2.5.1 Sub Thematic area 1....4
2.5.2 Sub Thematic area 2....4
2.5.3 Sub Thematic area n....4
2.6 Related Works and Gaps in the Literature....5
2.7 A Summary of the Related Works....5
2.8 Chapter Summary....6
3 RESEARCH METHODOLOGY....7
3.1 Introduction....7
3.2 Research Design....7
3.3 Baseline Study (Optional Section for Computer Science Project)....7
3.3.1 Population of the Study....7
3.3.2 Sample Size and Sampling Technique....7
3.3.3 Data Collection Methods....7
3.3.4 Instruments for Data Collection....7
3.3.5 Questionnaire....7
3.3.6 Data Analysis....8
3.4 System Design and Implementation (Required Section for Computer Science Project, about 15 pages)....8
3.4.1 Proposed Business Process....8
3.4.2 Requirements Specification....8
3.4.3 Design Specification....8
3.5 System Implementation (Required Section for Computer Science Project)....8
3.5.1 System Development....8
3.5.2 System Deployment....8
3.6 Chapter Summary....8
4 RESULTS....9
4.1 Introduction....9
4.2 Baseline Study Results (Optional Section for Computer Science Project)....9
4.3....9
4.3.1 Descriptive Statistics....9
4.3.2 Hypothesis Testing (Inference Statistics) / Thematic Analysis....9
4.4 System Automation and Implementation Results (Required Section for Computer Science Project)....10
4.4.1 System Implementation Results....10
4.4.2 System Validation....10
4.5 Chapter Summary....10
5 DISCUSSION AND CONCLUSIONS....11

5.1 Introduction....11
5.2 Discussion....11
5.2.1 Objective 1 Discussion....11
5.2.2 Objective 2 Discussion....11
5.3 Conclusions....11
5.4 Recommendations....11
5.5 Future Works....11
5.6 Chapter Summary....11
REFERENCES....12
APPENDICES....13
Appendix 1 - Questionnaire....13

LIST OF TABLES

Table 1: Literature Review and Gaps (Give A Minimum of 10 Key Articles) 5
Table 2. Inventory Functional Requirement Table 9
Table 3. Media Functional Requirements Table 9
Table 4. Non-Functional Requirements Table 10
Table 5. Hardware Requirements Table 10
Table 6. Software Requirements Table 12

LIST OF FIGURES

Figure 1 .Business Process Model 7
Figure 2 System Overview Diagram 13
Figure 3. Technical Member Use Case Diagram 14
Figure 4. Class Diagram 15
Figure 5. Dashboard 17
Figure 6. Media Center 17
Figure 7. Inventory Dashboard 18

LIST OF ABBREVIATIONS

1 INTRODUCTION AND BACKGROUND

1.1 Introduction

This project involved the design and partial implementation of a Student Information System (SIS) integrated with Moodle to support student information management and related academic and administrative processes within a university context. The system was designed to cover key stages of the student lifecycle, including application and admission, registration, fee management, teaching and assessment, results publication, progression and graduation.

A key design consideration was the separation of responsibilities between the SIS and Moodle. The SIS was designed to serve as the authoritative source for official institutional records, including student identity, registration, financial records and official results, while Moodle was intended to support teaching activities, submissions and assessment evidence. The project also considered how information could be exchanged between the two systems while maintaining data consistency, access control and accountability.

The work focused on establishing the system's structure, defining user roles and workflows, implementing selected functionality and examining how the system could support reliable information management. The resulting implementation provides a foundation for further development, although some functionality and aspects of the Moodle integration remain incomplete.

1.2 Background

University administration involves several interconnected processes, including admissions, student registration, academic delivery, assessment, examinations, finance, student support and reporting. These processes depend on accurate information being available to the appropriate users at the appropriate time.

When information and processes are handled through disconnected systems, spreadsheets or manual procedures, maintaining consistent records and coordinating activities can become challenging. Users may experience delays in obtaining information, uncertainty about the status of requests, duplicate data entry and difficulties identifying responsibility for particular tasks. These challenges highlight the need for systems that support coordinated processes and provide clear records of actions and decisions.

Learning Management Systems (LMSs), such as Moodle, support online teaching, learning activities, assignment submissions and assessments. However, learning activities and official university

administration serve different purposes. For example, assessment evidence recorded in Moodle does not, by itself, constitute an officially approved university result. Similarly, a record of a payment transaction does not necessarily establish that all institutional requirements for registration have been satisfied.

An integrated Student Information System can help coordinate these processes by defining the responsibilities of each system and establishing controlled mechanisms for exchanging information. Such a system should also support appropriate access permissions, clear record statuses, traceable decisions and the identification and recovery of errors.

Against this background, the project focused on designing and partially implementing an SIS integrated with Moodle, with emphasis on student lifecycle management, clear system responsibilities and controlled information exchange.

1.3 Statement of the Problem

Student information management involves multiple academic and administrative processes that must operate consistently to support the student lifecycle. Where these processes are fragmented or poorly coordinated, maintaining accurate records, tracking responsibilities and exchanging information between systems can become difficult.

The use of Moodle for teaching and assessment introduces an additional requirement to distinguish learning activity and assessment evidence from official institutional records. Without clearly defined system responsibilities and controlled integration mechanisms, inconsistencies may arise between academic information and administrative records. Inadequate access controls and audit trails may also make it difficult to determine who performed an action, when it occurred and whether it was authorised.

The problem addressed by this project was the need for a structured Student Information System that could support key student lifecycle processes while maintaining the integrity of official records and providing a defined basis for integration with Moodle. The project therefore focused on designing and partially implementing a system with defined user roles, controlled workflows, appropriate access permissions and mechanisms for managing information exchange and tracking system activities.

1.4 Aim of the Study

The aim of this project was to design and partially implement a secure, usable and traceable Student Information System integrated with Moodle to support key stages of the student lifecycle while maintaining the SIS as the authoritative source of official institutional records.

1.5 Project Objectives

The project was guided by the following specific objectives:

1. To define the responsibilities of the SIS and Moodle and establish the lifecycles and controlled transitions of relevant institutional records.
2. To design and implement role-based user interfaces that provide access to information and system functions according to users' roles and responsibilities.
3. To design and implement mechanisms for exchanging information between the SIS and Moodle, with consideration for data consistency, error handling and recovery from integration failures.
4. To incorporate appropriate security, authorization and auditing mechanisms to protect sensitive information and support accountability for important institutional actions.
5. To design and implement user interfaces that communicate relevant information, record statuses and required actions clearly.
6. To apply software engineering practices, documentation and testing to support the reliability, maintainability and verification of the developed system.
7. To develop and demonstrate a working prototype of selected student lifecycle processes and the SIS–Moodle integration functionality.

1.6 Importance of the study

The study is important because it explored how a Student Information System integrated with Moodle could support the coordination of academic and administrative processes while maintaining the integrity and traceability of institutional information.

a) To the University: The project provides a basis for examining how student records and administrative workflows can be managed within a more structured system. The separation of official institutional records from learning activity evidence may help support data consistency, accountability and more reliable information management.

b) To students and applicants: The system was designed to support clearer access to information about application progress, registration requirements and other student-related processes. Such functionality could help users understand their current status and identify the actions required of them.

c) To academic and administrative staff: The project demonstrates how defined user roles and workflows can help clarify responsibilities and coordinate activities across academic and

administrative functions. The proposed integration with Moodle also provides a basis for reducing unnecessary duplication of information between systems.

d) To future developers and researchers: The project provides a foundation for further development and investigation of SIS–Moodle integration. Its system structure, documented design decisions and implementation work may support future improvements, testing and extension of the system.

1.7 Scope of the study

This study focused on the design and partial implementation of a Student Information System (SIS) integrated with Moodle within the University of Zambia (UNZA) context. The project covered key processes across the student lifecycle, including admissions, student records, course registration and progression, teaching and assessment evidence, examinations and official results management, student finance, student support, graduation and credentials, notifications, auditing and reporting.

The technical scope covered the development of a web-based system using a modular architecture, with a frontend, backend and database. The project also considered containerisation, automated testing and continuous integration to support consistent development and verification. Integration between the SIS and Moodle was designed around defined interfaces and mechanisms for reliable data exchange, error handling and recovery.

The study was limited to the features implemented and demonstrated during the project. Real student records, live payment processing, production deployment and real external-service credentials were excluded. Synthetic or demonstration data was used where real institutional data was unavailable. Full integration with Moodle Cloud remained outside the completed scope.

The system considered different categories of users, including applicants, students, lecturers, admissions officers, registrars, finance officers, student advisers, deans and system administrators. Access to information and system functions was intended to reflect each user's role and responsibilities.

1.8 Organization of the Report

The report is divided into five chapters as follows:

Chapter One introduces the study and presents the background, statement of the problem, aim, objectives, research questions, scope and significance of the study.

Chapter Two reviews relevant literature on Student Information Systems, Learning Management Systems, Moodle integration, access control, data integrity and user experience. It discusses existing approaches, key findings and gaps relevant to the project.

Chapter Three describes the methodology used in the design and implementation of the system. It presents the requirements analysis, system design approach, architecture, tools and technologies, implementation process, testing methods and ethical considerations.

Chapter Four presents the system analysis, design and implementation. It describes the implemented features, the approach to SIS–Moodle integration, access control, auditing and error handling. It also presents the available testing and demonstration results and discusses the extent to which the project objectives were achieved.

Chapter Five presents the conclusions drawn from the study and discusses the extent to which the objectives were achieved. It identifies the project's limitations and provides recommendations for future improvements, including further Moodle integration, additional functionality and possible production deployment.

1.9 Chapter Summary

This chapter introduced the study by presenting the background and problem addressed by the Student Information System integrated with Moodle. It outlined the aim and objectives of the project, defined the scope of the work undertaken, and identified the potential significance of the system to the university, students, applicants, academic and administrative staff, and future developers. The chapter also outlined the organisation of the remaining chapters of the report.

2 . LITERATURE REVIEW

2.1 Introduction

This chapter reviews scholarly literature and relevant technical standards concerning the design and development of a Student Information System (SIS) integrated with a Learning Management System (LMS), specifically Moodle, in a higher-education environment. The purpose of the review is to establish what is already known about student information management, digital academic administration, system interoperability, information security, workflow management and software quality. It also examines the limitations of existing approaches to identify the research gap addressed by this project.

Universities manage interconnected activities throughout the student lifecycle, including applications, admissions, registration, fee management, teaching, assessment, results processing, academic progression and graduation. These activities require accurate information to be shared among students, lecturers, admissions officers, finance personnel, academic administrators and other authorised users. When information is managed through disconnected applications or poorly coordinated processes, institutions may experience duplicated data, unclear record status, delayed decisions and difficulties establishing who authorised a particular action.

Student Information Systems support administrative activities by maintaining student records and facilitating processes such as admissions, registration and academic administration. Learning Management Systems, on the other hand, support the delivery and management of teaching and learning activities. Moodle, for example, supports online course activities, assessment and learner participation. Although these systems serve related purposes, they do not necessarily have the same responsibilities or rules for approving institutional records.

Research by Mtebe and Raisamo [1] demonstrates the importance of evaluating learning management systems within the context of higher education in Sub-Saharan Africa. Their study developed and validated a model for assessing LMS success using data from students taking Moodle-based courses at the University of Dar es Salaam. This regional evidence is relevant because the success of an educational information system depends not only on its technical availability but also on its quality and usefulness to its users.

Research on student management systems in East Africa also identifies managerial, organisational and technical considerations that influence implementation. Manyengo [2], in a systematic review, identifies leadership, staff capacity, ICT infrastructure, data utilisation, cybersecurity and user support as interconnected factors in effective student management system implementation. These findings suggest that the design of a student information system should consider institutional processes, user responsibilities and information security alongside software functionality.

The chapter is organised thematically according to the objectives of the project. It first examines authoritative data domains and student record lifecycles, followed by role-based access and user experience. It then discusses SIS–LMS interoperability, privacy and academic record integrity, integration reliability, software quality and the demonstration of an end-to-end system. Related studies are compared to identify limitations and establish how the proposed system responds to the research problem.

2.2 Background to the Study

2.2.1 Student Information Systems in Higher Education

A Student Information System is an information system designed to support the collection, storage, processing and retrieval of student-related information. Depending on the institution, its functions may include application processing, admission decisions, registration, programme management, student records, financial information, examination administration and reporting.

The purpose of an SIS extends beyond storing student details. It should help an institution maintain consistent information and support the administrative processes through which students progress. For example, an applicant who has been admitted must be represented correctly in the student records before the relevant registration processes can be completed. Similarly, a student's academic record must distinguish between a course they have applied to study, a course for which they have registered and a course they have completed.

The importance of digital student management is also reflected in research conducted in Zambia. Chansa Thelma et al. [3] examined the effect of management information systems on student academic performance in selected higher-learning institutions in Lusaka District. Their study used questionnaires and interviews involving 100 respondents across four institutions. The authors reported potential benefits in information access and decision-making, while also identifying unequal access, limited digital literacy and occasional system downtime as challenges. Although the study does not specifically evaluate an SIS–Moodle integration architecture, it provides relevant local evidence about the benefits and implementation challenges associated with educational information systems.

UNZA also provides an important institutional context for this project. Its public-facing systems include a Student Information System and an online application portal [4], [5]. Consequently, this study should not assume that the University of Zambia has no student information system. Instead, it examines the design of an integrated platform that demonstrates how student lifecycle workflows, controlled data exchange, access restrictions and recovery mechanisms can be brought together.

For this project, the SIS is intended to act as the authoritative system for institutional records such as student identity, programme enrolment, registration, financial records, approved results and awards. This means that official decisions must be governed by institutional rules rather than being determined automatically by an external learning or payment service.

2.2.2 Learning Management Systems and Moodle

A Learning Management System supports the planning, delivery and monitoring of learning activities. Typical LMS functions include course content delivery, assignment submission, online quizzes, learning resources, discussions and feedback. Moodle is an open-source LMS used in educational environments to support online and blended learning.

The value of an LMS is different from that of a student information system. An LMS primarily supports the teaching and learning process, while an SIS supports institutional administration and the maintenance of official student records. The systems therefore complement one another but should not be assumed to have identical responsibilities.

Mtebe and Raisamo [1] studied Moodle-based learning in a Sub-Saharan African university context and developed a model for assessing LMS success. Their work demonstrates the relevance of evaluating the quality and success of learning platforms within the conditions of higher education in the region. Kambolokoni [6] also examined the effectiveness of Moodle in promoting online learning at the University of Zambia. This local study is useful for understanding the educational role of Moodle, although it does not establish the technical requirements for integrating Moodle with an SIS.

For the proposed system, Moodle is responsible for teaching-related activities such as course participation, submissions and quizzes. It may provide learning evidence or assessment information to the SIS, but the existence of a mark in Moodle does not automatically mean that the mark has been approved as an official university result.

For example, a lecturer may enter assessment marks in Moodle. Those marks may still require validation, moderation or approval under institutional regulations before they can be published as official results. The SIS must therefore preserve the distinction between provisional learning evidence and an approved academic outcome.

2.2.3 Integration of Administrative and Learning Systems

Interoperability refers to the ability of separate systems to exchange information and use that information meaningfully. In higher education, interoperability is important because administrative and learning systems manage different but related parts of the student lifecycle.

The 1EdTech Consortium's Edu-API specification provides a technical approach for exchanging higher-education enterprise data between systems such as student information systems and learning platforms [7]. Its relevance to this project lies in the need to define clear interfaces and data structures instead of allowing systems to access one another's databases without controlled boundaries.

A useful integration must specify which system owns each type of information, what information can be exchanged, how records are matched and what happens when an exchange fails. For example, an SIS course offering and a Moodle course may refer to the same teaching activity but use different identifiers. The integration must maintain a reliable mapping between the two records.

Integration also introduces reliability concerns. An external system may become unavailable, a message may be delivered more than once, or one system may complete an operation while the other does not. AWS Prescriptive Guidance describes the transactional outbox pattern as a way of coordinating database changes with the delivery of events to other systems [8]. Although this is technical guidance rather than a peer-reviewed study, it provides a practical explanation of an integration pattern relevant to the proposed architecture.

The system developed in this project therefore uses controlled integration boundaries, versioned contracts, idempotent message handling and reconciliation. These controls are intended to reduce inconsistent records and provide a defined recovery path when external operations do not complete as expected.

2.3 Thematic Area 1: Authoritative Domains and Record Lifecycles

Related to Objective 1: To define authoritative domains and record lifecycles.

This thematic area examines how a university information system should determine which system owns official data and how records progress through defined stages. The objective is necessary because student administration involves several related domains, but not every system should be allowed to make every institutional decision.

2.3.1 Authoritative Data and System Responsibilities

In an integrated information environment, a system of record is the designated source for a particular category of authoritative information. Other systems may use or display that information, but they should not independently change it in ways that undermine the designated source of authority.

In the proposed architecture, the SIS is the authoritative source for student identity, registration, financial records, official results and awards. Moodle manages teaching-related activities, including course participation, submissions and quizzes. This separation is a project design decision informed by the different purposes of administrative and learning systems.

Edu-API [7] is relevant because it addresses the exchange of academic enterprise data across higher-education systems. However, interoperability does not mean that every connected system should have equal authority over the records it exchanges. Data ownership, validation and approval rules must still be defined by the institution.

Consider a student who completes an online quiz in Moodle. Moodle can record the quiz attempt and calculated score. However, the SIS should not treat that score as a final university result unless the applicable academic process has been completed. This distinction protects the integrity of the official student record.

The literature provides a basis for separating administrative and learning functions. The remaining design challenge for this project is to express that separation through enforceable permissions, data mappings and approval workflows.

2.3.2 Workflow States and Controlled Transitions

A workflow describes the sequence of activities through which a process progresses. A record lifecycle describes the states a record can occupy and the conditions under which it can move from one state to another.

For example, an application may progress through the following states:

Draft → Submitted → Under Review → Approved or Rejected.

A result may follow a different lifecycle:

Draft → Validated → Approved → Published.

These states are not merely labels displayed on a screen. Each state represents a meaningful condition of the record. A submitted application should not be editable in the same way as an unfinished draft, and a published result should not be changed through an ordinary editing operation without the appropriate authority and audit history.

Workflow controls should therefore define who can perform an action, what conditions must be satisfied and what changes the action causes. These rules should be enforced by the backend rather than relying only on the interface to prevent invalid actions.

This approach is consistent with design-oriented information systems research. Peffers et al. [9] describe a design science research methodology in which a problem is identified, solution objectives are defined, an artefact is developed and demonstrated, and the artefact is evaluated. Their methodology is relevant to a project that develops a system artefact to address a practical information management problem.

2.3.3 Audit History and Preservation of Institutional Records

An audit trail records important actions performed on a system, including who performed an action, when it occurred and what was changed. Auditability is particularly important when a change affects admissions, registration, finance or official academic results.

If an official result is overwritten without preserving its previous state, it may be difficult to establish how the result changed or whether the change was authorised. Preserving an audit history supports accountability and helps administrators investigate errors or disputes.

For this project, high-impact actions should preserve an appropriate history rather than silently replacing earlier information. Permissions, record states and audit history should operate together: a user must have the appropriate authority to perform a transition, and the system should record the resulting change.

Research gap and project response. The sources reviewed establish the importance of interoperability and structured information system development. However, these sources alone do not demonstrate how the specific combination of authoritative SIS records, controlled lifecycle transitions and audit history performs in this project's university workflow. This project addresses that design question by implementing explicit record states, guarded transitions and auditable actions, then testing the behaviour of those controls.

2.4 Thematic Area 2: Role-Based Access and User Experience

Related to Objective 2: To implement role-based and scope-controlled user experiences.

2.4.1 Role-Based Access Control

Role-Based Access Control (RBAC) is an approach in which permissions are assigned to roles and users receive permissions through their assigned roles. This provides a structured alternative to granting access separately and informally to every user.

Ferraiolo, Barkley and Kuhn [10] describe an RBAC model and a reference implementation for networked web servers. Their work demonstrates how role-based rules can be used to manage and enforce access permissions in web-based information systems. Sandhu et al. [11] also present a model of RBAC intended to provide a unified foundation for role-based access control.

These studies are relevant to university systems because users have different responsibilities. Applicants submit applications, admissions officers review them, lecturers manage teaching activities,

finance officers handle financial records and authorised academic officers approve results. Giving every user unrestricted access would expose sensitive information and allow actions outside their responsibilities.

The proposed system therefore assigns role-specific permissions. However, role assignment alone may not be sufficient. A lecturer should not automatically access every course simply because they have the lecturer role. Access may also depend on the user's relationship to a particular course, department or record.

2.4.2 Scope-Controlled Permissions and Separation of Duties

Scope-controlled access restricts a user's permissions according to the records and activities they are authorised to manage. The system considers not only what role the user holds, but also which particular records fall within their responsibilities.

For example, an admissions officer may be authorised to review applications but should not necessarily be able to publish official examination results. A lecturer may access assessment information for assigned courses but should not be able to alter another department's records without authorisation.

Separation of duties is also important for high-impact actions. The user who initiates a process may not be the appropriate person to approve it. This reduces the risk of unauthorised changes and supports institutional accountability.

RBAC research [10], [11] provides a foundation for managing permissions through roles and constraints. The proposed system extends these principles at the application-design level by considering role, record scope, relationship and workflow state when evaluating actions.

The implementation must enforce these checks on the server. Hiding a button from an unauthorised user is not sufficient because a user may attempt to invoke an API endpoint directly.

2.4.3 Usability, Accessibility and Understandable Feedback

A secure system must also be usable. Users need to understand their current status, the actions available to them and the reasons an action cannot proceed. If a system displays a generic error without explaining what needs to happen next, users may repeatedly attempt the same action or contact support unnecessarily.

Mtebe and Raisamo [1] show the relevance of evaluating LMS quality and success within Sub-Saharan higher education. Their research supports the need to consider user experience and

system quality when developing educational information systems. Nguyen et al. [12] also developed and evaluated design principles for learning analytics information systems in higher education, illustrating the value of systematic design and evaluation of systems intended to support educational users.

In the proposed system, the interface should display a meaningful status, explain relevant restrictions and identify the next step where possible. For example, instead of displaying only "Registration failed," the system might explain that registration cannot proceed because a required approval has not been recorded.

Accessibility also matters because users may have different devices, levels of digital literacy and internet conditions. Interface design should use clear labels, consistent navigation, understandable validation messages and suitable keyboard and screen-reader support.

Research gap and project response. Existing access-control models explain how permissions can be structured, while educational information systems research examines system quality and user experience. The project brings these concerns together by implementing role- and scope-specific workspaces, server-side permission checks and status messages that explain the user's current position in a workflow. Their effectiveness must be evaluated through usability and authorisation tests.

2.5 Thematic Area 3: SIS–Moodle Interoperability and Integration

Related to Objective 3: To integrate Moodle and external providers through governed contracts.

2.5.1 Interoperability and Data Exchange

Interoperability enables separate systems to exchange data through agreed structures and interfaces. In higher education, this allows administrative systems and learning platforms to share information without requiring one system to duplicate all the functions of another.

The Edu-API specification [7] is particularly relevant because it addresses academic enterprise data exchange in higher education. It provides a reference for designing structured exchanges between student administration and learning systems.

A controlled integration should specify the data being exchanged, the identifiers used to match records, the meaning of each field and the responsibilities of each system. These agreements form the basis of an integration contract.

For example, an SIS may identify a course offering using an institutional code, while Moodle identifies a course using a different internal identifier. A reliable integration must maintain a mapping

between these identifiers. Without that mapping, an enrolment intended for one course could be applied to the wrong Moodle course.

2.5.2 API Contracts and Versioning

An Application Programming Interface (API) allows software systems to communicate through defined operations and data structures. API contracts specify the inputs, outputs and expected behaviour of those operations.

Versioning helps manage changes to an interface over time. If an integration changes the structure of an enrolment message, for example, an older consumer should not silently interpret the new message incorrectly. Versioned contracts allow changes to be introduced and tested deliberately.

The project uses defined integration contracts instead of direct cross-database editing. This separates the SIS's internal data model from Moodle's internal data model and makes it possible to validate data before applying changes.

The approach is especially important when an integration involves sensitive or consequential data. A malformed message should be rejected or placed into a controlled error-handling process rather than silently changing an official student record.

2.5.3 Data Ownership and Result Approval

Interoperability should not remove the distinction between learning evidence and official academic outcomes. Moodle may provide quiz scores, assignment marks or participation information, but the institution determines the rules for validation, moderation, approval and publication of official results.

The SIS–Moodle boundary must therefore define which data can be received from Moodle and what further actions are required before that data affects an official record.

For example, an incoming assessment result may be matched to a student and a course, checked for completeness and made available to an authorised academic officer. Publication should occur only after the applicable approval requirements have been satisfied.

Research gap and project response. Edu-API offers a relevant interoperability reference, but the existence of an API specification does not by itself solve local mapping, approval and governance questions. This project addresses those implementation concerns by defining the SIS as the owner of official records and Moodle as the source of teaching-related evidence, with validation and controlled transitions between them.

2.6 Thematic Area 4: Privacy, Data Integrity and Academic Governance

Related to Objective 4: To protect privacy, financial integrity and official results.

2.6.1 Privacy and Protection of Student Information

Student information systems may contain personal identifiers, contact information, programme details, academic records and financial information. Access to this information should be limited to authorised users who require it for legitimate responsibilities.

Role-based access control provides a technical basis for restricting access to data and operations [10], [11]. However, privacy protection also requires appropriate handling of information, secure authentication, careful logging and avoidance of unnecessary data exposure.

Pardo and Siemens [13] discuss ethical and privacy principles for learning analytics. Their work is relevant because educational systems can process detailed information about student behaviour and performance. It supports the need to consider transparency, responsible data use and privacy when designing systems that connect administrative records with learning-related information.

For this project, access should be limited according to user role and scope. Development and demonstration should use synthetic data rather than real student records, consistent with the project's stated scope.

2.6.2 Financial Integrity and Transaction Validation

Financial information can affect whether a student is permitted to register or proceed with particular administrative activities. Therefore, the system must distinguish between receiving evidence of a payment and making an authorised institutional decision based on that evidence.

A payment provider's callback is not, by itself, proof that every institutional condition has been satisfied. The system must validate the transaction and apply the relevant institutional rules before updating the student's financial status.

In the project prototype, financial processing is simulated. This allows the workflow and its validation rules to be demonstrated without processing real payments. Any future operational deployment would require approved institutional procedures, provider integration and additional security controls.

2.6.3 Official Results and Auditability

Official academic results have consequences for progression, graduation and institutional reporting. They therefore require controls that distinguish draft marks from validated, approved and published results.

Audit history supports the investigation of changes by recording relevant information about the action, actor and time. Combined with restricted permissions, it can help an institution determine whether an action was performed by an authorised user.

The proposed system applies these principles to result-related workflows. Moodle may supply assessment evidence, but official result publication remains governed by the SIS and the authorised institutional process.

Research gap and project response. The reviewed access-control and learning-analytics literature provides a foundation for privacy and controlled data use. The project applies these principles across administrative, financial and academic workflows, with particular attention to server-side enforcement, audit history and the separation of provisional evidence from official decisions.

2.7 Thematic Area 5: Integration Reliability, Error Recovery and Reconciliation

Related to Objective 5 in the integration-reliability aspect of the project: To ensure reliable external communication and recovery.

2.7.1 Reliability Challenges in Integrated Systems

When two systems communicate, the exchange may fail for several reasons. A service may be unavailable, a network request may time out, a message may be delivered more than once, or the receiving system may reject the information.

A system that handles only successful exchanges is incomplete. It must also define what happens when an operation fails after some work has already been completed.

For example, the SIS may record an approved enrolment but fail to communicate it to Moodle because Moodle is temporarily unavailable. The SIS should not lose the enrolment record, and the integration should provide a controlled way to retry the exchange and determine whether Moodle eventually received it.

2.7.2 Transactional Outbox and Idempotency

The transactional outbox pattern is designed to reduce inconsistencies that occur when a database update and the publication of a corresponding event are separate operations [8]. Instead of relying on two independent actions succeeding together, the application records the business change and an event for delivery within a controlled database transaction. A separate process then delivers the event to the external system.

Idempotency is another important integration concept. An operation is idempotent when repeating the same request does not produce additional unintended effects. This matters because retries can result in duplicate messages.

For example, if the same enrolment event is delivered twice, the receiving system should recognise that the event has already been processed rather than creating duplicate enrolments.

These patterns support reliable communication, but they do not remove the need to handle invalid messages, permanent failures or mismatched records. The system must still record failed operations and provide a way to investigate and resolve them.

2.7.3 Reconciliation and Recovery

Reconciliation is the process of comparing records or processing outcomes across systems to identify differences. It can help detect situations in which the SIS records an enrolment but Moodle does not, or where a message was delivered but the response was not recorded correctly.

A recovery process should distinguish between temporary failures, which may be resolved by retrying, and permanent failures, which require investigation or corrective action. Repeated retries should not continue indefinitely without recording the problem.

The proposed system uses controlled event delivery, idempotent handling and reconciliation as part of its integration design. These mechanisms are intended to make failures visible and recoverable instead of allowing inconsistent records to remain undetected.

Research gap and project response. The transactional outbox pattern provides implementation guidance, but its suitability must be demonstrated within the specific SIS–Moodle workflow. The project therefore treats failure handling as a testable system requirement, including duplicate delivery, unavailable services, retry behaviour and reconciliation.

2.8 Thematic Area 6: Software Quality, Testing and Maintainability

Related to Objective 6: To apply engineering and delivery controls.

2.8.1 Software Quality and Maintainable Architecture

A student information system contains multiple related functions and must remain understandable and maintainable as requirements change. Software architecture helps organise these functions and define how they interact.

The project uses a modular monolith architecture. This allows related functions to be organised into modules while keeping deployment and application operation comparatively straightforward for the prototype. Clear module boundaries can help separate admissions, registration, finance, academic records, permissions and integration responsibilities.

The choice of architecture should be justified by the project requirements and constraints rather than by assuming that one architecture is always superior. A modular monolith can provide clear internal boundaries without introducing the additional operational complexity of independently deployed services.

2.8.2 Automated Testing and Verification

Software testing provides evidence that implemented functions behave according to their requirements. For an SIS, testing must cover more than whether a page loads or a record can be saved.

Functional tests can verify that an application moves through the correct states. Authorisation tests can verify that permitted actions succeed and prohibited actions are rejected. Integration tests can examine whether information is transferred and mapped correctly between the SIS and Moodle. Failure tests can examine how the system behaves when an external service is unavailable.

The design science research methodology proposed by Peffers et al. [9] includes demonstration and evaluation of the developed artefact. This supports the need to demonstrate the system and assess whether it addresses the identified problem.

In this project, automated testing is intended to provide evidence for important behaviours, including API validation, access-control enforcement, integration recovery and end-to-end workflows. Docker Compose supports consistent local development environments, while continuous integration and automated end-to-end tests help make the demonstration repeatable.

2.8.3 Testing High-Impact Workflows

High-impact workflows require particular attention because errors may affect admissions, financial status or academic records. Tests should cover both successful and unsuccessful scenarios.

For example, a test should confirm that an authorised academic officer can approve a result when the required conditions are satisfied. Another test should confirm that an unauthorised user cannot publish the same result by directly calling an API endpoint.

Integration tests should also confirm that repeated events do not create duplicate records and that failed external exchanges can be identified and retried safely.

Research gap and project response. Design-oriented research supports the development and evaluation of software artefacts. The project applies that approach by linking each objective to demonstrable system behaviour and test evidence, rather than relying only on descriptions of implemented screens.

2.9 Thematic Area 7: Accessible Interfaces and End-to-End Demonstration

Related to Objective 7: To demonstrate an end-to-end working vertical slice.

2.9.1 Task-Focused Interfaces

A task-focused interface presents users with the information and actions relevant to their responsibilities. This can be more effective than providing every user with the same dashboard and expecting them to navigate through unrelated functions.

For example, an applicant needs to submit an application and view its status. An admissions officer needs to review the application and record a decision. A lecturer needs access to assigned courses and assessment activities. The interface should reflect these different tasks while maintaining the same underlying rules for authorisation and record state.

The work of Nguyen et al. [12] on design principles for learning analytics information systems demonstrates the value of systematic design and evaluation in higher-education information systems. Their study used a design science approach and evaluated the developed principles across four courses involving 1,173 students. Although the research focuses on learning analytics rather than a complete SIS, it provides a relevant example of how design principles can be developed and evaluated in an educational context.

2.9.2 End-to-End Workflow Demonstration

A system may contain several working modules without demonstrating that they operate together correctly. An end-to-end demonstration tests a connected workflow from its beginning to its final outcome.

For this project, the demonstration follows the student lifecycle through application submission, admissions review, student conversion, registration, simulated financial processing, Moodle integration, controlled result release and audit or recovery.

This approach helps establish whether the different modules preserve consistent information and enforce the intended rules across the workflow. It also provides a basis for comparing the implemented behaviour against the project objectives.

2.9.3 Scope and Limitations of the Demonstration

The project is a prototype and does not claim to be a production deployment of UNZA's official systems. It uses synthetic demonstration data, simulated finance and a controlled Moodle integration approach. Full operational integration, live payment processing, real student records and production deployment remain outside the stated scope.

These limitations matter when interpreting the results. A successful demonstration can show that the design and prototype behave as intended under tested conditions. It cannot, by itself, establish that the system is ready for institution-wide use or that it improves institutional outcomes at scale.

Research gap and project response. The literature supports the importance of educational information system design, usability and evaluation. This project contributes a demonstrable implementation that brings administrative workflows, controlled integration, access restrictions and recovery behaviour together within one prototype.

2.10 Related Works and Gaps in the Literature

The reviewed works address different parts of the problem of managing student information and supporting digital learning in higher education. Their contributions are useful, but their focus and scope differ from the system developed in this project.

Mtebe and Raisamo [1] developed and validated a model for evaluating LMS success in Sub-Saharan African higher education. Their research is relevant to understanding Moodle and the quality of learning systems in the region. However, it does not establish how a complete student lifecycle should be governed across an SIS and Moodle.

Manyengo [2] reviewed digital student management systems in East African higher education and identified managerial practices associated with their implementation. This provides regional evidence about the importance of infrastructure, staff capacity, cybersecurity and user support. However, the review does not directly demonstrate the particular integration controls implemented in this project.

Chansa Thelma et al. [3] investigated management information systems in selected higher-learning institutions in Lusaka District. Their findings concerning system access, digital literacy and downtime are relevant to the Zambian context. Nevertheless, the study focuses on MIS and student academic performance rather than the detailed design of SIS–Moodle integration contracts and recovery mechanisms.

Kambolokoni [6] examined Moodle's effectiveness in promoting online learning at UNZA. This provides local context for the educational role of Moodle, but the study does not establish how Moodle learning evidence should be mapped to, validated within and approved as part of official SIS records.

Edu-API [7] provides a technical specification for higher-education data exchange. It is useful for designing structured integration between administrative and learning systems. However, a standard does not automatically determine institution-specific policies for academic approval, result publication, user permissions or recovery from every failure scenario.

Ferraiolo, Barkley and Kuhn [10], together with Sandhu et al. [11], provide foundational work on role-based access control. These works explain how permissions can be organised and enforced through roles. The proposed project applies these principles to a university context where permissions may depend on both the user's role and their scope of responsibility.

The transactional outbox pattern described by AWS [8] provides a practical approach for reliable event delivery. It supports the project's integration design, although it is technical guidance rather than a peer-reviewed evaluation of the complete proposed system.

2.10.1 Identified Research Gaps

The comparison of these works suggests several areas that require further investigation.

1. **Institutional workflow gap.** Local and regional studies examine SIS use, student management and Moodle effectiveness, but the sources reviewed so far do not establish how all relevant student lifecycle stages can be connected through explicit, controlled record transitions in the specific context of this project.
2. **Authority and approval gap.** Interoperability enables data exchange, but the project must also establish which system owns each record and which institutional process authorises a provisional learning mark to become an official result.
3. **Access-control gap.** RBAC research provides a foundation for managing permissions. The project must demonstrate how roles, record scope and workflow state work together to prevent unauthorised actions within university processes.
4. **Integration-recovery gap.** Reliable delivery patterns provide ways to handle messages and retries. The project must demonstrate how these mechanisms behave when Moodle is unavailable, events are duplicated or records cannot be reconciled.
5. **Evaluation gap.** The existence of separate modules does not prove that a complete workflow behaves correctly. The project must provide test evidence for the connected journey and the security, integrity and recovery rules enforced throughout it.

These gaps are provisional and limited to the sources reviewed at this stage. They should be refined after the wider literature search, especially once more peer-reviewed SIS–LMS integration studies and relevant African university case studies have been examined.

2.10.2 Contribution of the Current Project

The project responds to these identified needs by designing and demonstrating an SIS–Moodle platform with explicit system responsibilities, controlled record lifecycles, role- and scope-based access, governed integration contracts, audit history and integration recovery.

Its contribution is not the claim that UNZA lacks an SIS or that no previous system integrates administrative and learning functions. Rather, the project demonstrates a particular combination of workflow governance, access control, integration reliability and testable end-to-end behaviour within a bounded prototype.

The project evaluates this contribution through functional, authorisation, integration and end-to-end tests. The resulting evidence will be used to determine which objectives have been achieved and which aspects require further development.

2.11 Literature Review and Gaps Table

Table 2.1: Summary of selected related works and their relevance to the project

Author/source	Topic and approach	Main contribution or finding	Limitation and relevance to this project

Mtebe and Raisamo [1]	Evaluation of LMS success in Sub-Saharan higher education; model validation using Moodle users	Developed and validated an LMS success assessment model using a survey of 200 students	Focuses on LMS success rather than full SIS–Moodle lifecycle integration
Manyengo [2]	Systematic review of digital student management systems in East Africa	Identifies leadership, staff capacity, infrastructure, data use, cybersecurity and user support as important implementation factors	Does not directly evaluate the specific integration architecture developed in this project
Chansa Thelma et al. [3]	MIS and academic performance in selected Lusaka higher-learning institutions; mixed methods	Reports potential benefits of MIS alongside challenges involving access, digital literacy and downtime	Provides local context but does not specify SIS–Moodle integration controls
UNZA Student Information System [4]	Public-facing institutional student system	Demonstrates that UNZA already provides student/staff access to an SIS	Public pages do not establish the system's internal architecture or integration limitations
UNZA Online Application Portal [5]	Public-facing online admissions portal	Demonstrates that applicants can access online application functions	Does not provide evidence about the complete internal admissions workflow
Kambolokoni [6]	Study of Moodle effectiveness at UNZA	Provides local research on Moodle's role in online learning	Does not focus on SIS integration or official result governance
1EdTech Edu-API [7]	Higher-education interoperability specification	Provides a standardised reference for academic enterprise data exchange	Does not determine all institution-specific approval, ownership and recovery rules
AWS Prescriptive Guidance [8]	Transactional outbox technical pattern	Explains reliable event delivery and the need to manage duplicate processing	Technical guidance; the project must test how the pattern behaves in its own workflows

Peffers et al. [9]	Design science research methodology	Provides a structured process for identifying a problem, developing an artefact, demonstrating it and evaluating it	Methodological foundation rather than a study of SIS–Moodle integration
Ferraiolo, Barkley and Kuhn [10]	RBAC model and web implementation	Establishes a formal approach to managing permissions through roles	Requires adaptation to the project's record scope and workflow rules
Sandhu et al. [11]	Unified model of RBAC	Provides a formal foundation for role-based permissions and constraints	Does not independently define university-specific permission policies
Nguyen et al. [12]	Design principles for learning analytics information systems in higher education	Developed and evaluated design principles through design science research	Focuses on learning analytics rather than the complete administrative student lifecycle
Pardo and Siemens [13]	Ethical and privacy principles for learning analytics	Discusses privacy and ethical responsibilities when using educational data	Provides principles rather than an implementation of the project's complete security architecture

2.12 **Chapter Summary**

This chapter reviewed literature relevant to the development of a Student Information System integrated with Moodle. The review examined student information systems in higher education, Moodle's role in teaching and learning, interoperability between administrative and learning systems, workflow management, role-based access control, privacy, academic record integrity, integration reliability and software testing.

The literature indicates that student information systems and learning management systems support related but distinct institutional responsibilities. A successful integration must therefore address not only data exchange but also data ownership, record validation, approval procedures and the protection of official academic information. Research on access control provides a basis for restricting user permissions, while integration patterns provide practical approaches to managing message delivery, duplicate processing and failures.

Zambian and Sub-Saharan African studies provide relevant context concerning the use of educational information systems, Moodle, infrastructure, digital literacy and system quality. However, the sources reviewed at this stage do not fully establish how the particular combination of controlled student lifecycle workflows, role- and scope-based permissions, SIS–Moodle integration and tested recovery behaviour performs within the proposed prototype.

The project addresses this design and implementation question by developing a bounded system that maintains the SIS as the authority for official institutional records and uses controlled integration to exchange relevant information with Moodle. The system will be evaluated through tests of functional behaviour, access restrictions, data integrity, integration recovery and end-to-end workflows. The next chapter presents the methodology, design approach, tools, implementation process and evaluation strategy used to develop and assess the prototype.

3 METHODOLOGY

3.1 Introduction

This chapter describes the research methodology and software engineering methods used in the development of the Student Information System (SIS) integrated with Moodle. It explains how the problem was investigated, how the system requirements were identified, how the system was designed and implemented, and how the resulting prototype was evaluated.

Since this is a Computer Science and Software Engineering project, the methodology covers two related activities. The first is the research process used to understand the problem, identify the system requirements and evaluate the proposed solution. The second is the software development process used to design, implement, test and deploy the prototype.

The study adopted Design Science Research Methodology (DSRM), supported by descriptive evaluation. DSRM was selected because the main output of the project is a software artefact intended to address an identified information management problem. The project therefore involves more than describing the challenges associated with student administration; it requires the development and evaluation of a working system.

The prototype was developed using a modular software architecture and a technology stack consisting of Next.js, TypeScript, NestJS, PostgreSQL and Prisma. Docker Compose was used to support a reproducible local development and demonstration environment. Automated testing tools were used to evaluate system functionality, access control, integration behaviour and selected failure scenarios.

The system was demonstrated using synthetic data, with financial processing simulated and Moodle integration restricted to the defined prototype scope. The project did not process real student records or live payments, and it was not deployed as a production system.

The chapter is organised into research design, baseline process analysis, requirements specification, system design, implementation and deployment, evaluation procedures, ethical considerations and a chapter summary.

3.2 **Research Design**

3.2.1 Design Science Research Methodology

Design Science Research Methodology is an approach to research in which an artefact is designed and developed to address an identified problem. The artefact may be a software application, system architecture, model, method or other designed solution.

Peffers et al. [1] proposed a Design Science Research Methodology for Information Systems Research that provides a structured process for identifying a problem, defining solution objectives, designing and developing an artefact, demonstrating it, evaluating its performance and communicating the findings.

This methodology was appropriate for the present study because the main research output is a working SIS–Moodle prototype. The project aims to demonstrate how student lifecycle processes, access control, data ownership and system integration can be managed through a single, governed application architecture.

Unlike a study that relies primarily on questionnaires to measure people's opinions, this project evaluates a technical artefact. The evaluation therefore focuses on whether the system satisfies its requirements and whether important behaviours can be verified through functional tests, authorisation tests, integration tests and end-to-end demonstrations.

The six DSRM activities described by Peffers et al. [1] were adapted to the project as follows.

1. Problem identification and motivation

The first activity involved identifying the information management challenges addressed by the project. These included the need to coordinate student lifecycle activities, maintain clear ownership of institutional records, restrict sensitive operations to authorised users and exchange information with Moodle without confusing learning evidence with official academic records.

The problem was investigated through the review of relevant literature, analysis of the target workflows and examination of available system and technical documentation. The project did not assume that the University of Zambia had no existing SIS. Instead, it focused on designing and demonstrating an integrated approach to student administration and learning-system interaction.

2. Definition of solution objectives

The second activity involved translating the identified problem into specific objectives for the proposed system. The objectives covered authoritative data domains and record lifecycles, role- and scope-based access, governed integration, protection of sensitive records, accessible interfaces, software engineering controls and an end-to-end working demonstration.

These objectives guided the selection of functional requirements, the design of system modules and the evaluation procedures.

3. Design and development

The third activity involved designing and implementing the system in manageable increments. Rather than developing every module independently and integrating them only at the end, the project used vertical slices that connected related functions across the student lifecycle.

A vertical slice represents a complete path through relevant parts of the system. For example, an application-submission slice connects the user interface, backend validation, database persistence, submission receipt and audit history. A later slice connects approved student registration with simulated financial processing and Moodle-related evidence.

4. Demonstration

The fourth activity involved demonstrating the developed functions using fictional student records and controlled test scenarios. Demonstrations were designed to show how users complete tasks, how records move through defined states and how the system responds to successful and unsuccessful operations.

The demonstration scope included applicant workflows, admissions review, registration, simulated financial processing, Moodle-related information exchange and controlled result publication.

5. Evaluation

The fifth activity involved checking whether the developed system behaved according to its requirements. Evaluation methods included functional testing, API testing, browser-based end-to-end testing, authorisation allow/deny tests, selected integration-failure tests and audit-history verification.

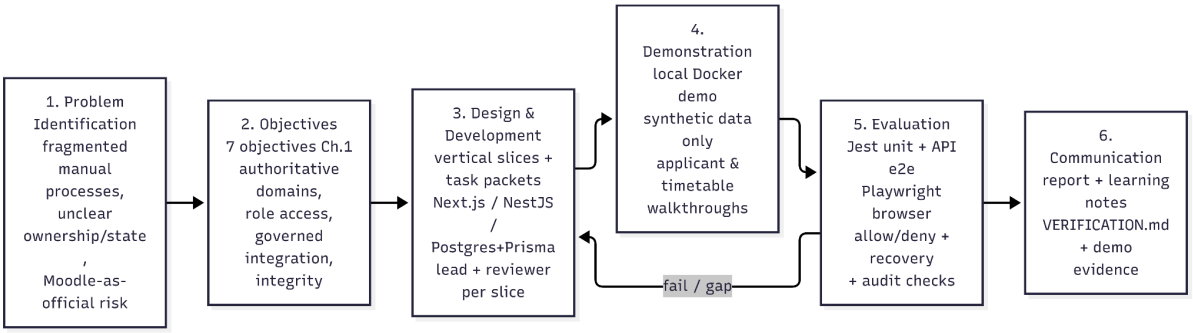
The evaluation was intended to establish evidence of system behaviour rather than rely only on the appearance of completed interfaces.

6. Communication

The final activity involved documenting the problem, methodology, requirements, system architecture, implementation decisions and evaluation approach in the project report. Technical artefacts, test records and demonstration evidence provide additional support for explaining how the system was developed and assessed.

The DSRM activities were iterative. Findings from testing could require a requirement, implementation or design decision to be revised before the affected functionality was considered complete.

Figure 3.1: Design Science Research process used in the project



3.2.2 Justification for the Research Design

DSRM was selected because it aligns with the project's main output and evaluation needs. The project is concerned with building a system that demonstrates particular technical and administrative behaviours, not merely describing an existing process.

For example, the objective of preventing unauthorised result publication can be evaluated by testing whether an authorised user can complete the operation and whether an unauthorised user is rejected. Similarly, the integration objective can be examined by testing how the system handles an incoming Moodle-related record, a duplicate event or an unavailable external service.

These behaviours provide measurable evidence that can be used to evaluate the prototype against its requirements.

The study also uses descriptive evaluation to document the system's behaviour. Test outcomes can be reported as passed, failed or not tested, while usability observations, where collected, can be summarised descriptively. This does not mean that the study claims to establish the effect of the system on the academic performance of actual UNZA students. Such a claim would require a different research design and suitable field evidence.

3.3 Baseline Study of Current Processes

3.3.1 Purpose of the Baseline Study

The baseline study was used to understand the student administration activities that the proposed system needed to support. It established the target workflow, the actors involved, the information exchanged and the decisions that needed to be controlled.

The analysis focused on the following activities:

- Programme discovery and application submission.
- Admissions review, clarification and decision-making.
- Conversion of an admitted applicant into a student record.
- Course registration and registration-readiness checks.
- Financial record review and clearance.
- Exchange of relevant information with Moodle.
- Staging, validation, approval and publication of results.
- Notification, audit history and recovery from failed operations.

The baseline was informed by the project requirements, relevant literature and available institutional or technical documentation. The workflows were then represented as system processes to be supported by the prototype.

It is important to distinguish this type of baseline analysis from a formal investigation of all existing UNZA administrative practices. The project did not use a large-scale staff questionnaire or claim that every listed activity is currently performed manually at UNZA. Where internal procedures or implementation details were unavailable, the proposed workflow was treated as a design requirement rather than a verified statement about the university's existing operations.

3.3.2 Population of the Study

In a conventional survey, the population consists of the people or units from which observations are collected. This project did not conduct a population-based questionnaire survey. Its primary unit of analysis was the set of student administration processes and system interactions represented in the prototype.

The scope of the analysis included three main categories.

Administrative processes: These included application submission, admissions decisions, registration, financial processing, result approval and publication.

System actors: These included applicants, students, admissions officers, finance officers, lecturers, academic advisers and authorised administrators. These roles were used to define the responsibilities and permissions that the system needed to support.

Information artefacts and technical components: These included application records, registration records, financial records, Moodle-related evidence, result records, audit events, API contracts and test records.

The study therefore focused on process and system behaviour rather than estimating the views of a wider human population.

3.3.3 **Sampling Technique and Selection of Process Slices**

Purposive selection was used to identify representative process slices for development and evaluation. Purposive selection involves choosing cases or activities because they are particularly relevant to the research problem.

This approach was appropriate because the project aimed to demonstrate the interaction of key system components across the student lifecycle. The selection was based on functional importance, the need to enforce institutional rules and the need to test interactions between modules.

Three major process slices were selected.

Slice 1: Programme discovery and application submission

This slice covered the discovery of available programmes, creation of an application draft, completion of required sections, submission and generation of a submission receipt. It was selected to demonstrate data validation, ownership of application records and protection against duplicate submission.

Slice 2: Admissions review and student registration

This slice covered application review, requests for clarification, recording of an admissions decision, acceptance and conversion of an admitted applicant into a student record. It also included registration-related checks. This slice was selected to demonstrate controlled workflow transitions and role-based permissions.

Slice 3: Financial processing, Moodle evidence and result publication

This slice covered simulated financial processing, exchange or retrieval of Moodle-related evidence, staging of results, validation, approval, publication and audit or recovery operations. It was selected because it crosses multiple system boundaries and involves high-impact institutional records.

The process slices were not intended to represent a statistically sampled population. They were selected to provide functional coverage of the principal workflows and the technical controls relevant to the project's objectives.

3.3.4 Data Collection Methods

The project used secondary information and system-based evidence to identify requirements and evaluate the prototype.

Document and literature review: Scholarly publications, relevant technical documentation and available institutional materials were reviewed to understand SIS functions, Moodle, access control, data exchange, reliability and software development practices.

Process analysis: The selected workflows were broken down into actors, inputs, decisions, state transitions, outputs and failure conditions. This analysis helped identify what each module needed to do and which rules had to be enforced.

Technical inspection: The design and implementation artefacts were examined to establish how the frontend, backend, database, integration adapter and testing tools supported the required behaviours.

Software testing: Automated test runs, browser walkthroughs, verification checklists and audit records were used to assess whether the implemented functions behaved according to the requirements.

No large-scale questionnaire was used as the principal data collection method. Any future usability survey would need to be reported separately, including the participants, procedure and results, if such an evaluation is actually conducted.

3.3.5 Research Instruments

The project used several instruments to organise development and evaluation.

Task packets: These described the functionality to be implemented, the relevant workflow, expected behaviour and acceptance conditions.

Requirements and verification checklists: These were used to confirm that the relevant functional and non-functional requirements had been addressed.

Demonstration scripts: These described the steps needed to demonstrate selected workflows, including expected successful outcomes and important restrictions.

Automated test suites: Jest was used for appropriate unit and API tests, while Playwright was used for browser-based end-to-end tests. These tools supported repeatable verification of application behaviour.

Test and audit records: These provided evidence of operation outcomes, including successful actions, rejected actions, integration failures and relevant history entries.

These instruments were selected because the project required traceable evidence of software behaviour rather than only opinions about the proposed system.

3.3.6 Data Analysis

The evaluation data consisted primarily of test outcomes and observations of system behaviour.

Functional tests were analysed according to whether the expected behaviour occurred. For example, a test of application submission checked whether the system validated the application, persisted the required information and generated the expected submission record.

Authorisation tests were analysed by comparing the result with the expected permission decision. An operation that should be allowed was expected to succeed for an appropriately authorised user, while an operation outside that user's permissions was expected to be rejected.

Integration tests examined whether the system handled the expected data correctly and whether failures were visible and recoverable. Relevant scenarios included duplicate events, unavailable external services and records requiring reconciliation.

Audit verification examined whether important operations produced the required history and whether restricted information remained appropriately protected.

The findings were described using test status and supporting evidence. Where quantitative summaries were available, they could include the number of tests passed, failed and not executed. Such figures should be taken from actual test results rather than estimated or created for the report.

Usability feedback, if collected, could be summarised using response frequencies and descriptive statistics. However, no survey findings are claimed in this methodology unless a corresponding evaluation is actually performed.

3.4 System Design and Implementation

This section explains how the identified requirements were translated into the architecture, modules, database structures, workflows and security controls of the proposed SIS–Moodle platform.

The design was guided by seven project objectives:

1. Define authoritative data domains and record lifecycles.
2. Implement role-based and scope-controlled user experiences.
3. Integrate Moodle and external providers through governed contracts.
4. Protect privacy, financial integrity and official results.
5. Provide accessible and understandable interfaces.
6. Apply engineering, testing and delivery controls.

7. Demonstrate a working end-to-end vertical slice.

The following subsections explain the main design decisions and their relationship to these objectives.

3.4.1 Proposed Business Process

The proposed business process represents the intended flow of student information through the system. It connects administrative activities while ensuring that each operation follows the appropriate rules and approval requirements.

The process begins when an applicant discovers an available programme and creates an application. The applicant completes the required information and submits the application. The system validates the submission, records it and provides a receipt.

The application then enters an admissions review process. An authorised admissions officer examines the application and may request clarification or record a decision. If the applicant is accepted and completes the required acceptance process, the system can create or activate the appropriate student record.

The student can then proceed through registration, subject to the applicable academic and financial checks. Financial information is handled through the defined finance workflow. In the prototype, financial processing is simulated rather than connected to a live payment provider.

Moodle-related information is handled through a controlled integration boundary. Teaching evidence can be received or retrieved within the supported scope, mapped to the appropriate student and course records, and checked before being used in a result workflow.

Result publication follows a separate sequence involving staging, validation and authorised approval. The system records the relevant actions and makes published results available to appropriately authorised users.

3.4.2 Requirements Specification

Requirements specification describes what the system must do and the conditions under which it must operate. The requirements were derived from the project objectives, analysis of the selected workflows, relevant literature and technical considerations.

Requirements were divided into functional requirements, non-functional requirements, hardware requirements and software requirements.

3.4.2.1 Functional Requirements

Functional requirements describe the operations and services provided by the system.

Table 3.1: Functional requirements of the SIS–Moodle prototype

Functional requirement	Description

Authentication and sessions	The system shall allow users to sign in and maintain an authenticated session.
Role and scope enforcement	The system shall restrict actions and information according to user role, record scope and applicable workflow rules.
Programme discovery	The system shall allow users to view and search available programme offerings.
Application management	Applicants shall be able to create, save, validate and submit applications.
Submission confirmation	The system shall generate a submission receipt and preserve the relevant submission record.
Admissions review	Authorised admissions officers shall be able to review assigned applications, request clarification and record decisions.
Student conversion	The system shall support the conversion of an accepted applicant into a student record according to defined rules.
Registration	The system shall support course or section selection and apply the required registration checks.
Financial record management	The system shall support the demonstration of financial records and reconciliation using simulated finance processing.
Moodle integration	The system shall support the defined, controlled exchange or retrieval of Moodle-related information within the prototype's scope.
Result staging and validation	The system shall support staging and validating assessment evidence before it becomes an official result.
Result approval and publication	The system shall restrict official result approval and publication to authorised users.
Notifications	The system shall provide relevant status notifications to authorised users.
Audit history	The system shall preserve a history of selected high-impact operations and decisions.

Integration recovery	The system shall provide mechanisms to identify and recover from supported integration failures.

The functional requirements define the expected system capabilities. Detailed acceptance criteria and test cases should be linked to each requirement to establish how its implementation will be verified.

3.4.2.2 Non-Functional Requirements

Non-functional requirements describe the qualities and operational constraints of the system.

Table 3.2: Non-functional requirements

Requirement	Description
Security	The system shall enforce authentication, server-side authorisation, input validation and least-privilege access.
Reliability	The system shall handle supported integration failures through controlled retries, idempotency and reconciliation.
Data integrity	The system shall validate data and prevent unauthorised or invalid state transitions.
Auditability	The system shall retain appropriate history for important administrative, financial and academic actions.
Maintainability	The system shall use clear module boundaries and consistent programming conventions.

Usability	Interfaces shall use clear labels, meaningful status information and understandable error messages.
Accessibility	The interface shall support appropriate keyboard navigation and other relevant accessibility practices.
Compatibility	The application shall be designed for testing in supported modern web browsers.
Reproducibility	The local development and demonstration environment shall be reproducible using the defined container configuration.
Privacy	The prototype shall use synthetic data and restrict access to information according to user permissions.

These requirements guide the technical design and testing. Where a requirement involves a measurable performance or accessibility target, the final report should specify the target and explain how it was tested. The table does not claim that a performance target has been achieved unless corresponding measurements are available.

3.4.2.3 Hardware Requirements

Hardware Requirements

The prototype was designed for local development and demonstration rather than production deployment.

Table 3.3: Hardware requirements

Hardware component	Minimum or recommended specification	Purpose
Development computer	Intel Core i5 or equivalent processor, or better	Running the development environment and application services
Memory	8 GB minimum; 16 GB recommended	Running the frontend, backend, database, containers and tests
Storage	Approximately 20 GB of available space recommended	Storing source code, container images, database volumes and test artefacts
Display and input devices	Standard monitor, keyboard and mouse	Development, testing and browser demonstration
Internet connection	Required when downloading dependencies or accessing external resources	Dependency installation and any supported external integration testing

These specifications are practical recommendations for the local prototype, not the result of formal hardware benchmarking. Actual requirements depend on the size of the development environment and the services running simultaneously.

3.4.2.4 Software Requirements

Table 3.4: Software requirements

Software or technology	Purpose
Next.js	Frontend web application and user interface
TypeScript	Type-safe development across the frontend and backend
CSS Modules and Tailwind CSS where used	Styling and presentation of the user interface
NestJS	Backend API and application business logic
PostgreSQL	Relational database management
Prisma	Database schema management and data access

Docker Compose	Local orchestration of application and database services
Node.js LTS	Runtime environment for the TypeScript application
Jest	Unit and appropriate API-level testing
Playwright	Browser-based end-to-end testing
GitHub Actions	Automated build and test checks within the development workflow

The technologies were selected to support a structured, maintainable web application. TypeScript provides a common programming language across the frontend and backend. NestJS supports the organisation of backend functionality into modules, while PostgreSQL provides relational data storage and Prisma supports database access and schema management.

Docker Compose supports a reproducible local environment. Jest and Playwright support testing at different levels, from individual functions to complete browser-based workflows. GitHub Actions can automate selected checks so that changes are evaluated before they are accepted into the main development branch.

3.4.3 Design Specification

The design specification describes the main architectural components and the models used to represent system behaviour and information. It includes the high-level architecture, logical modules, use cases, activity diagrams, class design, sequence diagrams, database design and security design.

3.4.3.1 High-Level System Architecture

The system uses a modular monolith architecture. In this approach, the backend is organised into modules with defined responsibilities, but the application is deployed as a unified backend rather than as a collection of independently deployed microservices.

The main architectural components are:

Presentation layer: The Next.js frontend provides the user interface for applicants, students, admissions officers, finance staff, lecturers and authorised administrators.

Application and API layer: The NestJS backend handles requests, validates inputs, applies business rules, checks permissions and coordinates database operations.

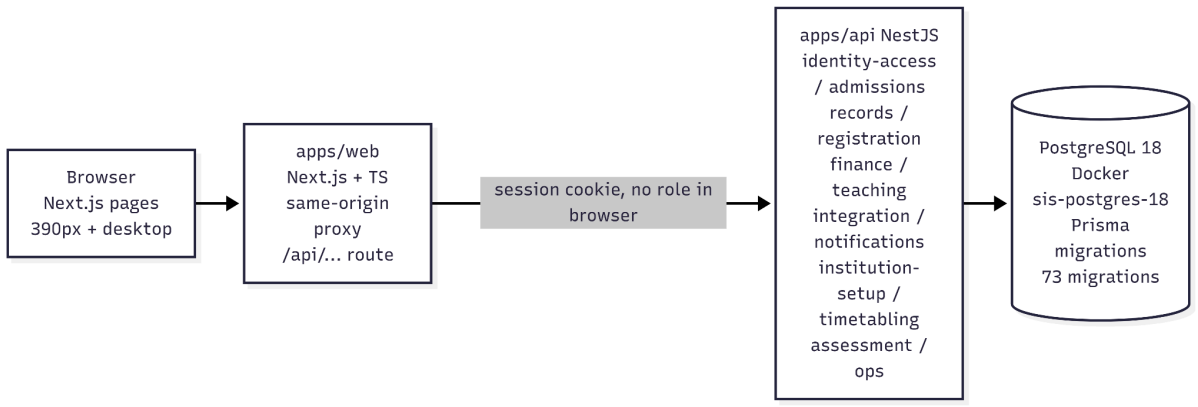
Domain modules: These include admissions, registration, finance, Moodle integration, results, notifications and audit-related functionality. Each module is responsible for a defined part of the student lifecycle.

Data layer: PostgreSQL stores application records, student records, registration information, financial records, mappings, result information and audit-related data. Prisma supports structured access to the database.

Integration boundary: A dedicated adapter handles the supported communication with Moodle. It applies the relevant mapping and validation rules instead of allowing the frontend or unrelated modules to edit Moodle's database directly.

Development and verification environment: Docker Compose supports local execution of the application services, while automated tests verify selected application and integration behaviours.

Figure 3.3: High-level architecture of the proposed system



The architecture separates presentation, application logic, data storage and external integration. This separation makes it easier to apply business rules consistently and prevents user-interface code from becoming the sole enforcement point for security or workflow restrictions.

3.4.3.2 Logical Design and Module Boundaries

The logical design defines the responsibilities of each module and how modules interact.

The admissions module manages application creation, submission, review, clarification and decisions. The registration module manages the transition from an admitted applicant to a registered student and supports relevant registration checks.

The finance module manages the prototype's financial records and reconciliation workflow. Because live payment processing is outside the project scope, financial operations are simulated.

The Moodle adapter manages the defined exchange or retrieval of learning-related information. It is responsible for mapping external identifiers, validating incoming information and recording integration outcomes.

The results module manages the staging, validation, approval and publication of results. It must ensure that Moodle-related evidence does not become an official result without the required institutional process.

The notification module communicates relevant status changes, while the audit functionality records selected high-impact operations.

These module boundaries are intended to reduce unnecessary coupling. For example, the results module should not need to know how a Moodle API request is transported; it should receive validated evidence through a defined internal interface.

3.4.3.3 Use Case Diagrams

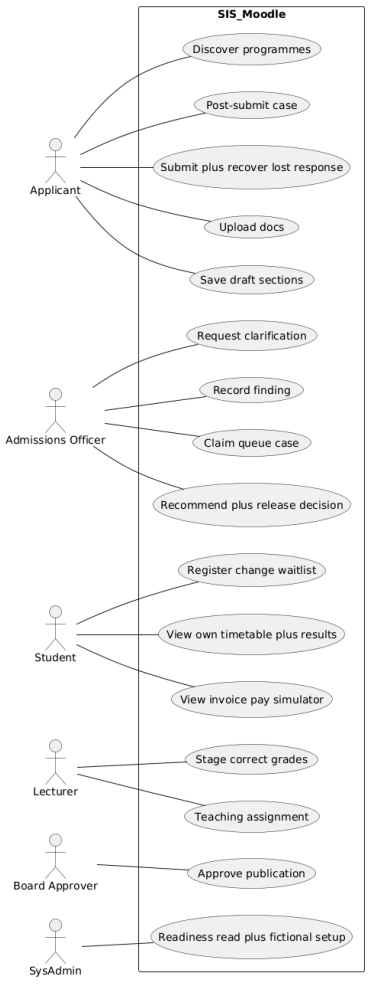
Use case diagrams represent the interactions between system actors and the functions they are permitted to perform.

The principal actors in the prototype are the applicant, student, admissions officer, finance officer, lecturer and authorised academic administrator.

The applicant interacts with programme discovery, application drafting, application submission and application-status functions. The admissions officer reviews applications, requests clarification and records decisions. The student interacts with registration and personal academic information. Finance personnel work with the supported financial-record and reconciliation functions.

Lecturers or other authorised academic users may work with the supported Moodle-related evidence and relevant academic processes. An authorised academic officer approves and publishes official results.

Figure 3.4: Main use case diagram



The purpose of the use case model is to define system boundaries and actor responsibilities. Detailed permission checks are specified separately because the ability to access a use case may also depend on record scope and workflow state.

3.4.3.4 Activity Diagram

Activity diagrams describe the sequence of steps, decisions and alternative paths within a process.

Three activity diagrams are particularly relevant to the system.

Application drafting and submission: The diagram begins when an applicant creates a draft. The applicant completes required sections, and the system validates the data. If required information is missing, the application remains a draft, and the system displays the relevant blockers. If the application satisfies the submission rules, the system records the submission and generates a receipt.

Admissions clarification and decision: The diagram begins when an authorised officer opens an application. The officer reviews the information and either requests clarification or records the appropriate decision. A clarification request returns the application to the applicant for the required response before the review continues.

Result staging, approval and publication: The diagram begins when supported assessment evidence is received or retrieved. The system checks the mapping and stages the evidence. The result is then validated and submitted for approval. If approval is rejected or validation fails, publication does not occur. If the required conditions are satisfied, an authorised action publishes the result and the system records the outcome.

Figure 3.5: Activity diagrams for key workflows

These diagrams help identify decision points and invalid transitions before implementation. They also provide a basis for constructing test cases for both successful and unsuccessful paths.

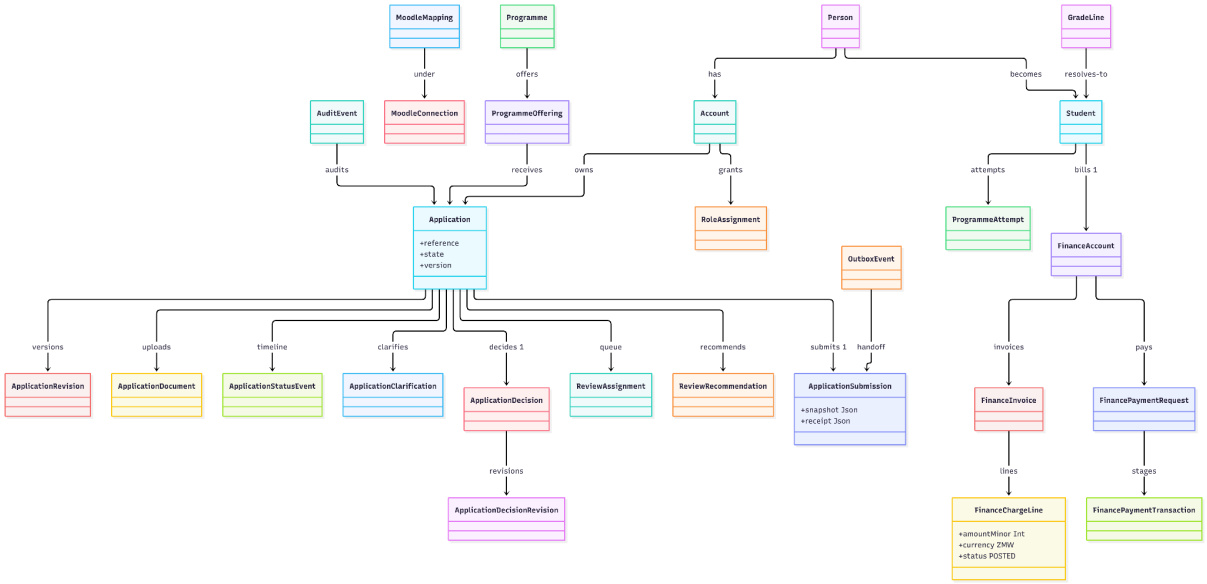
3.4.3.5 Class Diagram

A class diagram describes the principal data entities and their relationships.

The core entities include:

- **Application:** Represents an applicant's application and its current state.
- **ApplicationRevision:** Preserves relevant versions of application information.
- **Submission:** Represents the submission event and its confirmation.
- **Decision:** Records the outcome of an admissions decision.
- **Student:** Represents the student record created or maintained after admission.
- **Registration:** Represents a student's registration information.
- **Invoice or FinancialRecord:** Represents the financial information managed by the prototype.
- **MoodleMapping:** Links institutional records to their corresponding Moodle identifiers.
- **ResultStaging:** Represents assessment evidence awaiting validation or approval.
- **AuditEvent:** Records selected actions and their relevant context.
- **OutboxEvent:** Represents an event awaiting delivery to an external system, where the transactional outbox mechanism is implemented.

Figure 3.6: Conceptual class diagram



The diagram should show that an application may have multiple revisions and that relevant decisions are associated with the application. A student may have registration records, while staged results are linked to the appropriate student and course mapping. Audit events should be associated with the relevant action or record where appropriate.

The final class diagram must be consistent with the actual Prisma schema. Any entity that was not implemented should be marked as part of the proposed design or omitted from the implementation diagram.

3.4.3.6 Sequence Diagrams

Sequence diagrams illustrate how components communicate over time to complete an operation.

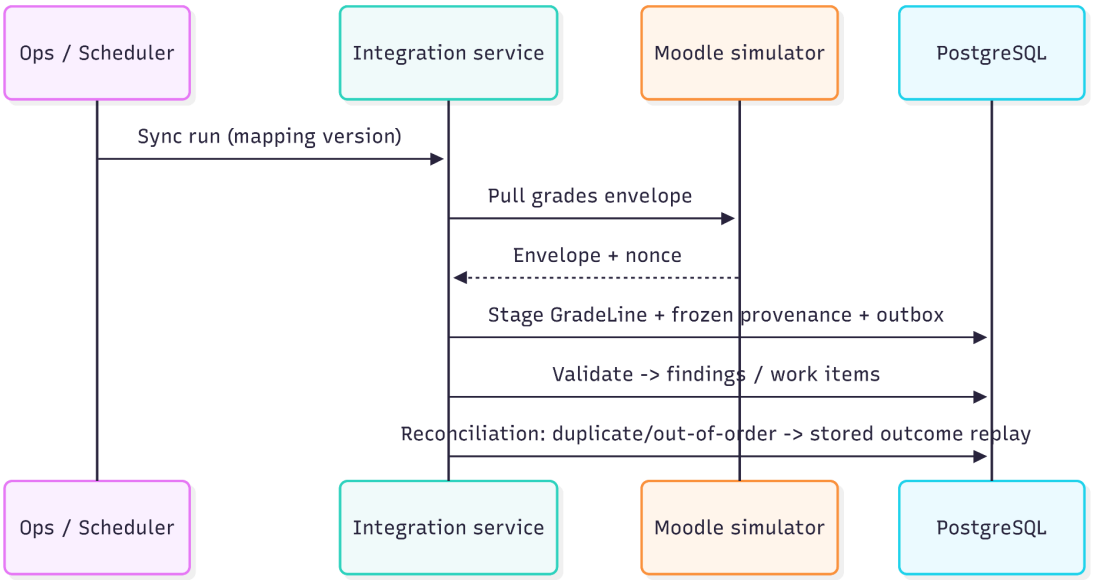
Three sequences are important for the project.

Application submission: The applicant submits an application through the frontend. The backend authenticates the user, checks permissions, validates the application and performs the required database operations. If the submission is successful, the system returns a receipt. Repeated submission requests must be handled according to the defined idempotency rules.

Submission recovery after a lost response: The frontend submits a request, but the response is lost or times out. The applicant retries or checks the submission status. The backend must determine whether the original operation was completed and return the appropriate outcome without creating an unintended duplicate submission.

Moodle evidence retrieval and reconciliation: The integration adapter requests or receives supported Moodle-related information, validates the response and maps external identifiers to institutional records. If the operation fails, the system records the failure and applies the supported retry or reconciliation process.

Figure 3.7: Sequence diagrams for submission and integration



These diagrams help explain the order of operations and identify where validation, transactions, idempotency and audit recording are required.

3.4.3.7 Database Design

The system uses PostgreSQL as its relational database. The database design organises related records into tables and defines relationships through primary keys, foreign keys and appropriate constraints.

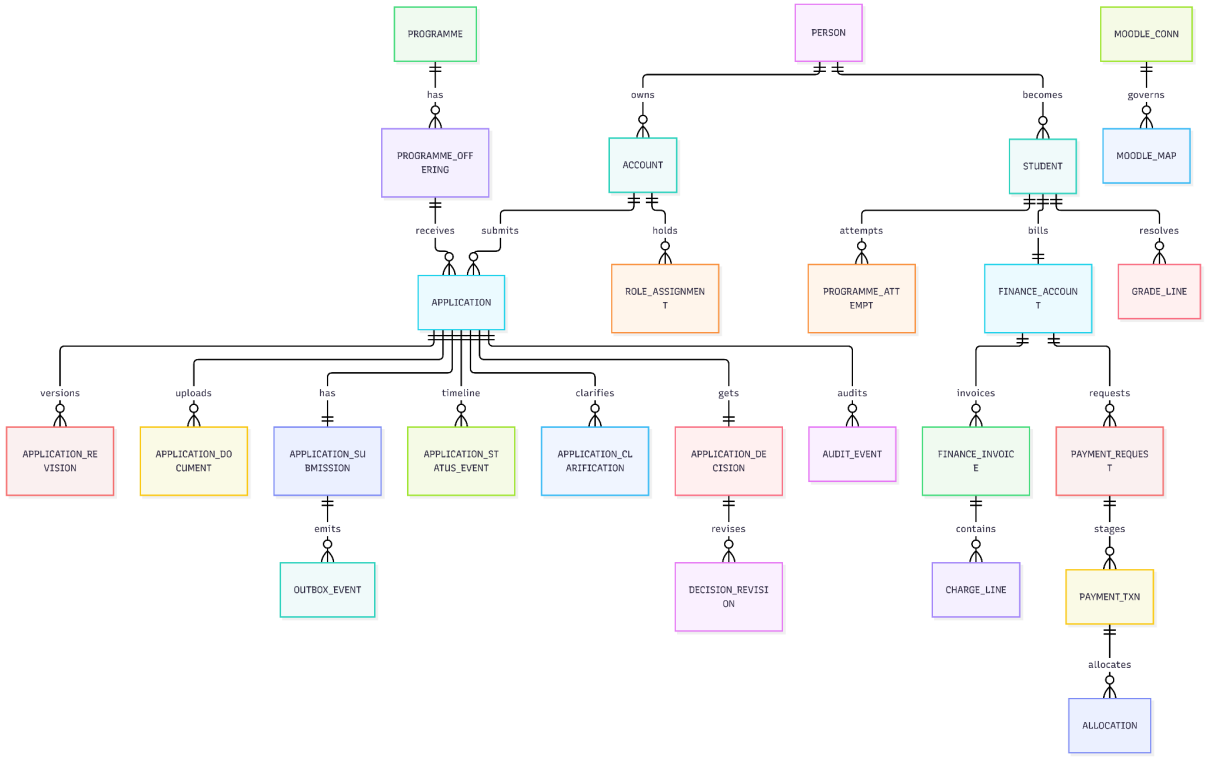
The schema must support the principal student lifecycle entities, including applications, application revisions, admissions decisions, students, registrations, financial records, Moodle mappings, staged results and audit events.

Data integrity is supported through validation in the application layer and appropriate database constraints. Important operations should use transactions where multiple related changes must succeed or fail together.

For example, an application submission may require the creation of a submission record, a receipt and relevant audit information. If an outbox event is also required, it should be persisted consistently with the business operation. This reduces the risk of recording a successful submission without recording the associated event that must be delivered to another component.

The database design must also preserve the distinction between provisional and official information. A staged result should not be indistinguishable from a published result, and a financial record should not be treated as confirmed merely because a payment-related request was initiated.

Figure 3.8: Entity-relationship diagram



3.4.3.8 Security Design

Security was treated as a cross-cutting design requirement because the system manages sensitive student, financial and academic information.

Authentication and sessions: Users must authenticate before accessing protected functions. The server is responsible for validating the session and identifying the authenticated user.

Server-side authorisation: Permissions must be enforced by the backend. The system considers the user's role and, where applicable, the relevant record scope, relationship and workflow state.

Least privilege: Users should receive only the permissions needed to perform their assigned responsibilities. For example, a lecturer should not automatically have permission to publish official results for every course.

Input validation: Data received from forms and APIs must be validated before being used in business operations or stored in the database.

Session protection: Session-related security measures should include appropriate cookie settings and protection against unauthorised session use. Sensitive configuration values must not be exposed in client-side code.

Audit history: Important administrative, financial and academic actions should generate appropriate audit information. Access to audit information must also be controlled.

File handling: Where file uploads are implemented, files should be validated and handled through the defined storage and scanning controls. A quarantine and scanning process should be used where supported by the implementation.

Failure handling: Where an operation cannot be verified or a required service is unavailable, the system should avoid silently treating the operation as successful. High-impact actions should fail safely when required checks cannot be completed.

These controls are intended to reduce the risk of unauthorised changes, accidental data exposure and inconsistent official records. Their effectiveness must be evaluated through relevant tests rather than assumed from the design description alone.

3.5 System Implementation and Deployment

3.5.1 System Development Process

The system was developed incrementally, with related functionality implemented and verified in vertical slices. This allowed important end-to-end behaviour to be tested before the entire system was completed.

The development process followed these activities:

1. Define the task and the relevant acceptance conditions.
2. Identify the affected modules, data entities and permissions.
3. Implement the required frontend and backend functionality.
4. Apply database changes where required.
5. Run relevant unit, API and browser-based tests.
6. Verify permitted and prohibited actions.
7. Test applicable error-handling and recovery behaviour.
8. Record relevant implementation and verification evidence.

The frontend was developed using Next.js and TypeScript. It provides the user-facing pages and sends requests to the backend through the defined application interfaces.

The backend was developed using NestJS and TypeScript. It contains the business logic, validation, authorisation and module-level operations. PostgreSQL provides persistent data storage, while Prisma supports the application's interaction with the database.

The codebase was organised into application areas and shared contracts where appropriate. Shared contracts help maintain consistency between components that exchange structured data.

The implementation process gave particular attention to operations that could create or change important institutional records. For example, application submission required more than saving a form: it needed to validate the request, persist the submission and generate the relevant confirmation and history.

Similarly, the result workflow needed to distinguish between receiving assessment evidence, validating it, approving it and publishing it. These steps were treated as distinct operations rather than one unrestricted update.

3.5.2 Integration Implementation

The Moodle integration was designed around a defined adapter boundary. The adapter is responsible for the supported communication with Moodle and for mapping external information to the appropriate institutional records.

The integration was designed to avoid direct cross-database editing. This reduces coupling between the two systems and allows the application to validate incoming information before it is used in an institutional workflow.

Versioned contracts support controlled changes to the data exchanged between components. Idempotency mechanisms help prevent duplicate processing when an event or request is repeated. Retry and reconciliation mechanisms provide a way to identify and resolve supported integration failures.

The implemented integration scope must be distinguished from planned future functionality. The project notes specify a bounded, read-oriented Moodle integration and a prototype demonstration environment. Full production integration, unrestricted bidirectional synchronisation and live institutional deployment are outside the stated scope.

The final report should describe the exact Moodle integration path that was implemented and tested. If the demonstration used a simulator or limited read-only evidence, it should not claim that a complete live Moodle integration was achieved.

3.5.3 Testing During Development

Testing was carried out as part of the development process rather than being left entirely until the end.

Unit tests were used for appropriate individual functions and business rules. API tests were used to check backend operations, validation and permission enforcement. Playwright was used for browser-based workflows that involved interactions between the user interface and backend services.

Tests were designed to cover successful behaviour as well as important failure conditions. For example, a successful submission test checks that a valid application can be submitted. A related negative test checks that an incomplete application cannot be submitted if required information is missing.

Authorisation tests check that users can perform permitted operations but are prevented from performing actions outside their authority. Integration tests examine the handling of mapped information, duplicate events and selected failure scenarios.

The exact results of these tests should be reported in the evaluation section or implementation chapter using the actual test output. The methodology establishes how testing was conducted; it does not itself establish that every requirement passed.

3.5.4 Deployment and Execution Environment

The prototype was configured for local execution using Docker Compose. The local environment includes the web application, backend API and PostgreSQL database, with additional services included where required by the implementation.

Containerisation supports repeatability by providing a consistent way to start the required application services. It also reduces dependence on manually configuring each component separately on the development computer.

The intended demonstration process involves starting the configured services, loading the synthetic demonstration data and executing the selected workflows. Where a database reset script is provided, it can restore the demonstration environment to a known state for repeatable testing.

Automated checks may be executed through the development pipeline to identify build or test failures before changes are accepted. Backup and restore procedures can also be used to verify that the local demonstration database can be recovered.

The prototype was not deployed to a production university server. Its deployment claims are therefore limited to the local, reproducible environment that was configured and tested.

3.6 System Evaluation

3.6.1 Purpose of Evaluation

The purpose of evaluation was to determine whether the developed prototype satisfied the stated requirements and demonstrated the intended system behaviours.

Evaluation was aligned with the seven project objectives. The evaluation did not attempt to measure changes in actual university-wide performance, student academic outcomes or staff productivity because the prototype was not deployed as a live institutional system.

Instead, it focused on functional correctness, access control, record integrity, integration behaviour, interface clarity and end-to-end workflow completion.

3.6.2 Evaluation Methods

The following methods were used or specified for evaluating the prototype.

Functional testing: Checked whether system functions produced the expected outcomes for valid and invalid inputs.

Authorisation testing: Checked whether permissions were enforced according to the user's role and relevant record scope.

Integration testing: Checked whether the supported exchange of information followed the defined mapping and validation rules.

Failure and recovery testing: Checked selected scenarios involving unavailable services, duplicate requests or events, and incomplete exchanges.

Audit verification: Checked whether relevant high-impact operations generated the expected history.

End-to-end testing: Checked whether the selected workflow could progress across the relevant modules from its initial action to the expected outcome.

Usability and accessibility checks: Examined whether the interface presented clear status information and supported the applicable keyboard navigation and accessibility requirements.

Only methods actually performed should be reported as completed evaluation activities. Any checks that remain planned should be identified as planned or outstanding rather than presented as completed.

3.6.3 Evaluation Matrix

Table 3.5: Relationship between project objectives and evaluation methods

Project objective	Evaluation method	Example verification

Define authoritative domains and record lifecycles	Functional and state-transition tests	Confirm that a provisional Moodle-related mark cannot bypass the required approval workflow.
Implement role-based and scope-controlled access	Authorisation tests	Confirm that a user cannot access or change records outside their permissions.
Integrate Moodle through governed contracts	Integration tests	Confirm that incoming information is validated and mapped to the appropriate institutional record.
Protect privacy, financial integrity and official results	Security, integrity and audit tests	Confirm that restricted actions are rejected and high-impact changes are traceable.
Provide accessible and understandable interfaces	Interface inspection and accessibility checks	Confirm that status, validation errors and next steps are understandable and that relevant keyboard interactions work.
Apply engineering and delivery controls	Automated tests and build checks	Confirm that required checks execute and that failures are reported.
Demonstrate an end-to-end vertical slice	End-to-end tests and demonstration scripts	Confirm that the selected application-to-result workflow completes according to the defined rules.

The matrix provides traceability between objectives and the evidence required to evaluate them. It should be updated to reflect the actual test cases and results produced during implementation.

3.7 Ethical Considerations

Ethical considerations were addressed through the use of synthetic data, the limitation of the prototype's operational scope and the design of access controls.

3.7.1 Protection of Student Information

The prototype used fictional student records rather than real student information. This reduced the risk of exposing personal, financial or academic information during development and demonstration.

The project did not require access to real student credentials or confidential production databases. Any demonstration accounts were intended for the local test environment.

3.7.2 Financial and External-System Boundaries

Financial processing was simulated. The prototype did not collect or process live payments, and payment-related workflows were limited to demonstrating the relevant application behaviour.

Moodle integration was limited to the defined prototype scope. The project did not claim to make unauthorised changes to a live institutional Moodle system or directly modify an external system's database.

These boundaries reduced the risks associated with financial transactions and changes to external academic records.

3.7.3 Access Control and Accountability

Access to system functions and records was restricted according to the designed permission rules. Audit history was used to support accountability for important operations.

Audit records must themselves be protected because they may contain information about sensitive actions. Access to audit information should therefore be limited to authorised purposes and roles.

3.7.4 Responsible Reporting

The project report distinguishes between implemented features, tested behaviours, design intentions and future work. It does not claim production readiness, institution-wide adoption or measurable improvements in university performance without supporting evidence.

Similarly, any limitations discovered during evaluation should be reported honestly. This is necessary to ensure that the conclusions reflect what the prototype actually demonstrates.

3.8 Chapter Summary

This chapter described the research and software engineering methodology used to develop the SIS–Moodle prototype. Design Science Research Methodology was selected because the project produces a software artefact that is developed and evaluated against identified requirements.

The baseline analysis focused on the student lifecycle processes, actors and system interactions relevant to the project. Purposive selection was used to identify representative process slices covering application submission, admissions review, registration, financial processing, Moodle-related evidence and result publication.

The chapter also specified the functional and non-functional requirements, hardware and software environment, high-level architecture, logical modules, use cases, activity diagrams, class design, sequence diagrams, database design and security controls.

The implementation approach used incremental vertical slices and automated testing. Local deployment was supported through Docker Compose, with synthetic data and simulated financial processing. Evaluation was aligned with the project objectives and focused on functional correctness, authorisation, data integrity, integration reliability, interface behaviour and end-to-end workflow completion.

Finally, the chapter outlined the ethical safeguards used to protect information and limit the risks associated with the prototype. The next chapter should present the actual system implementation and

its results, including screenshots, selected technical details, test evidence and an assessment of the extent to which the project objectives were achieved.

4 M

REFERENCES

[1] J. S. Mtebe and R. Raisamo, "A model for assessing learning management system success in higher education in Sub-Saharan countries," *The Electronic Journal of Information Systems in Developing Countries*, vol. 61, no. 1, pp. 1–17, 2014, doi: 10.1002/j.1681-4835.2014.tb00436.x.

[2] P. R. Manyengo, "Digital student management systems in East African higher education: A systematic review of managerial practices," *Huria: Journal of the Open University of Tanzania*, vol. 33, no. 2, 2026, doi: 10.61538/huria.v33i2.2118.

[3] C. C. Thelma, B. A. Welbeck, D. L. Mpolomoka, C. Sylvester, and E. V. Phiri, "The effect of management information system on student academic performance: A case of selected higher learning institutions in Lusaka District, Zambia," *World Journal of Advanced Research and Reviews*, vol. 24, no. 3, pp. 118–129, 2024, doi: 10.30574/wjarr.2024.24.3.3669.

[4] University of Zambia, "Student Information System," 2026. [Online]. Available: https://sis.unza.zm/Users/login. [Accessed: Oct. 15, 2026].

[5] University of Zambia, "E-Campus Student Information System: Online Application Portal." [Online]. Available: https://ecampusonline.unza.zm/users/apply. [Accessed: Oct. 15, 2026].

[6] N. Kambolokoni, *The Effectiveness of Moodle in Promoting Online Learning in Tertiary Public Institutions: The Case of the University of Zambia*. Lusaka, Zambia: University of Zambia, 2023.

[7] 1EdTech Consortium, "Edu-API: Academic Enterprise Data for Higher Education." [Online]. Available: https://www.1edtech.org/standards/edu-api. [Accessed: Oct. 15, 2026].

[8] Amazon Web Services, "Transactional Outbox Pattern," *AWS Prescriptive Guidance*. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html. [Accessed: Oct. 15, 2026].

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[10] D. F. Ferraiolo, J. Barkley, and D. R. Kuhn, "A role-based access control model and reference implementation within a corporate intranet," *ACM Transactions on Information and System Security*, vol. 2, no. 1, pp. 34–64, 1999, doi: 10.1145/300830.300834.

[11] R. Sandhu, D. Ferraiolo, and D. R. Kuhn, "The NIST model for role-based access control: Towards a unified standard," in *Proceedings of the Fifth ACM Workshop on Role-Based Access Control*, 2000, doi: 10.1145/344287.344301.

[12] A. Nguyen, T. Tuunanen, L. Gardner, and D. Sheridan, "Design principles for learning analytics information systems in higher education," *European Journal of Information Systems*, vol. 30, no. 5, pp. 541–568, 2021, doi: 10.1080/0960085X.2020.1816144.

[13] A. Pardo and G. Siemens, "Ethical and privacy principles for learning analytics," *British Journal of Educational Technology*, vol. 45, no. 3, pp. 438–450, 2014, doi: